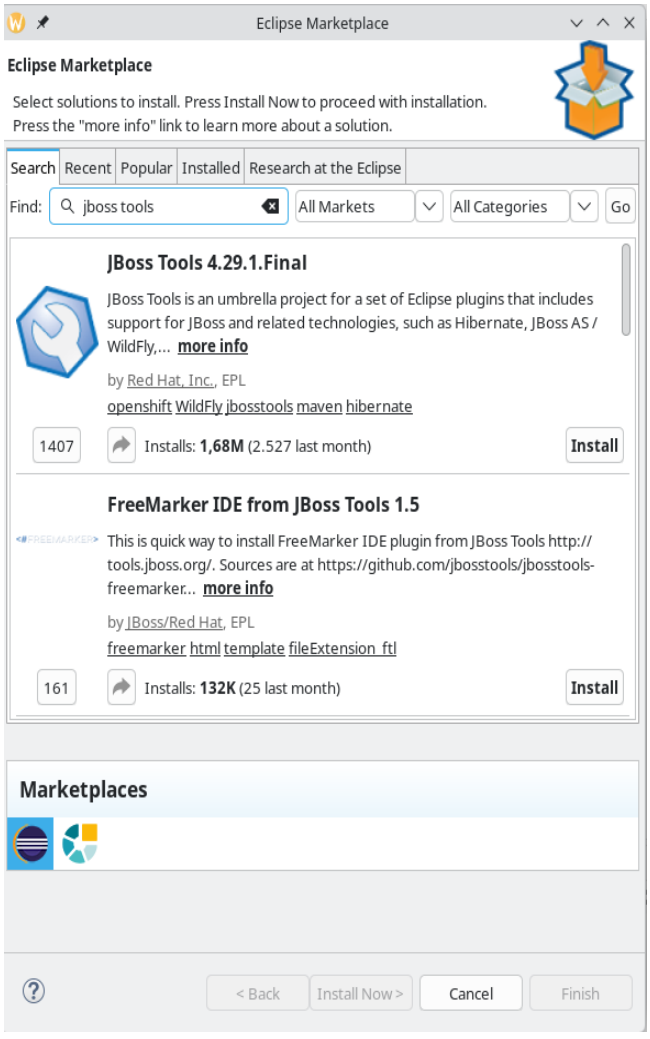
GENERACIÓN AUTOMÁTICA DE CLASES A PARTIR DE LAS TABLAS
Primera parte

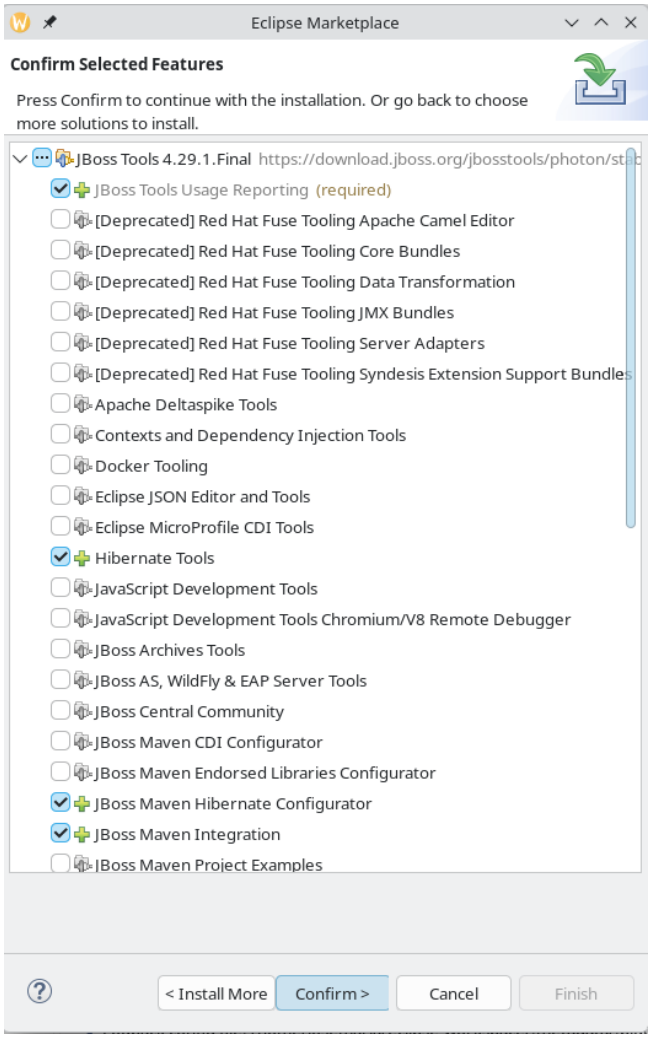
La primera parte de la configuración la hacemos una vez y no hay que volver a hacerla salvo que reinstalemos Eclipse.

Primero, instalamos las JBoss Tools. Entramos en

Help -> Eclipse Marketplace

Buscamos Jboss Tools (imagen de la izquierda), pulsamos Install y seleccionamos solo los cuatro elementos que aparecen marcados en la imagen de la derecha:





Después de pedirnos aceptación y confirmación un par de veces nos reiniciará Eclipse para finalizar la instalación

Segunda parte

Esta segunda parte la tendremos que hacer cada vez que creemos un proyecto nuevo en el que queramos usar esta herramienta de generación automática de entidades/clases a partir de las tablas de una base de datos ya existente. Nosotros vamos a trabajar aquí con la base de datos naciones

Creamos un proyecto Maven igual que hemos hecho en anteriores ocasiones. No olvides marcar la opción de Create a simple project y elige el nombre de tu group ID y artifact ID

Modificamos el archivo pom.xml así:

```
<project xmlns="http://apache.org"
	xmlns:xsi="http://w3.org"
	xsi:schemaLocation="http://apache.org https://apache.org">
	<modelVersion>4.0.0</modelVersion>
	<!-- Debe de corresponderse con el group ID y artifact ID que hayas
elegido tú -->
	<groupId>com.dam2.goya</groupId>
	<artifactId>adat-jpa3</artifactId>
	<version>1.0-SNAPSHOT</version>

	<properties>
		<maven.compiler.source>17</maven.compiler.source>
		<maven.compiler.target>17</maven.compiler.target>
		<project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
	</properties>

<dependencies>
		<!-- evita los mensajes de info de SLF4J -->
		<dependency>
			<groupId>org.slf4j</groupId>
			<artifactId>slf4j-nop</artifactId>
			<version>2.0.16</version>
		</dependency>
		<dependency>
			<groupId>org.hibernate.orm</groupId>
			<artifactId>hibernate-core</artifactId>
			<version>6.5.2.Final</version>
		</dependency>
		<dependency>
			<!-- No olvides cambiar esto si usas MySQL u otro gestor de
bases de datos -->
			<groupId>org.mariadb.jdbc</groupId>
			<artifactId>mariadb-java-client</artifactId>
			<version>3.3.3</version>
		</dependency>
	</dependencies>

	<build>
		<plugins>
```

```
                <plugin>
                        <groupId>org.hibernate.tool</groupId>
                        <artifactId>hibernate-tools-maven</artifactId>
                        <version>6.5.2.Final</version>
                        <executions>
                                <execution>
                                        <id>generate-entities</id>
                                        <phase>generate-sources</phase>
                                        <goals>
                                                <goal>hbm2java</goal>
                                        </goals>
                                </execution>
                        </executions>
                        <configuration>
                                <!-- En este directorio se dejaran las clases que
se generen -->

        <outputDirectory>${project.basedir}/src/main/java</outputDirectory>
                                <!-- Y el paquete se llamará así: -->
                                <packageName>model</packageName>
                                <ejb3>true</ejb3>
                                <jdk5>true</jdk5>
                                <properties>


<hibernate.connection.driver_class>org.mariadb.jdbc.Driver</hibernate.connection
.driver_class>
                                        <!-- Datos de conexión a la base de datos --
>

        <hibernate.connection.url>jdbc:mariadb://localhost:3306/naciones</
hibernate.connection.url>

        <hibernate.connection.username>profe</hibernate.connection.username>

        <hibernate.connection.password>abc123</hibernate.connection.password>


<hibernate.dialect>org.hibernate.dialect.MariaDBDialect</hibernate.dialect>

        <hibernate.default_catalog>naciones</hibernate.default_catalog>

        <hibernate.default_schema>naciones</hibernate.default_schema>
                                </properties>
                        </configuration>
                        <dependencies>
                                <dependency>
                                        <!-- No olvides cambiar esto si usas MySQL u
otro gestor de bases de datos -->
                                        <groupId>org.mariadb.jdbc</groupId>
                                        <artifactId>mariadb-java-client</artifactId>
                                        <version>3.3.3</version>
                                </dependency>
                        </dependencies>
```

```
                </plugin>
            </plugins>
        </build>
</project>
```

NOTA: pom.xml configurado para usar mariadb y la base de datos naciones con el resto de datos de nuestro proyecto de ejemplo. Atento a los comentarios para saber donde hay que personalizarlo para otros

En src/main/resources creamos el archivo hibernate.properties con el siguiente contenido:

```
hibernate.connection.driver_class=org.mariadb.jdbc.Driver
hibernate.connection.url=jdbc:mariadb://localhost:3306/naciones
hibernate.connection.username=profe
hibernate.connection.password=abc123
hibernate.dialect=org.hibernate.dialect.MariaDBDialect

hibernate.default_catalog=naciones
hibernate.default_schema=naciones
```

Igualmente, está personalizado para este ejemplo. Tendrás que hacer los cambios pertinentes para adaptarlo a otros casos.

A continuación añadimos JDBC como en cualquier proyecto. Para ello pulsamos con el botón derecho sobre el nombre del proyecto:

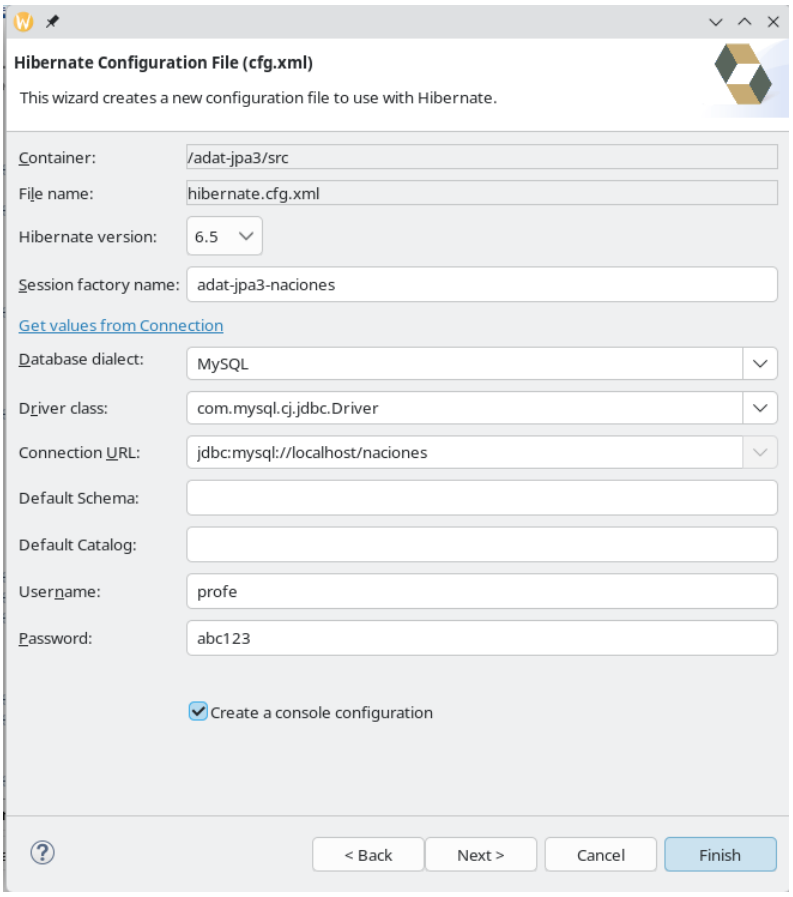
Build Path -> Add External Archive

Y elegimos el .jar del conector J como siempre de un directorio de nuestro disco donde no lo vayamos a eliminar o mover de ubicación

Ahora, sobre el proyecto pulsamos en

New -> Other -> Hibernate -> Hibernate Configuration File.

Crearemos el fichero en la carpeta src de nuestro proyecto y mantenemos el nombre que nos sugiere por defecto (hibernate.cfg.xml)



Los campos a rellenar son:

- **Elegimos un nombre para Sessionfactoryname: procura que tenga relación con el proyecto que realizas para no confundirte!**
- **Database dialect: MySQL**
- **Driver class: com.mysql.cj.jdbc.Driver. Cuidado porque esta opción no aparece en el desplegable y tenemos que corregirla nosotros.**
- **Connection URL: jdbc:mysql://localhost/naciones (lo último es el nombre de la DB)**
- **Username: un usuario que tenga acceso a la base de datos**
- **Password: la contraseña del anterior**
- **Marcamos Create a Console Configuration**

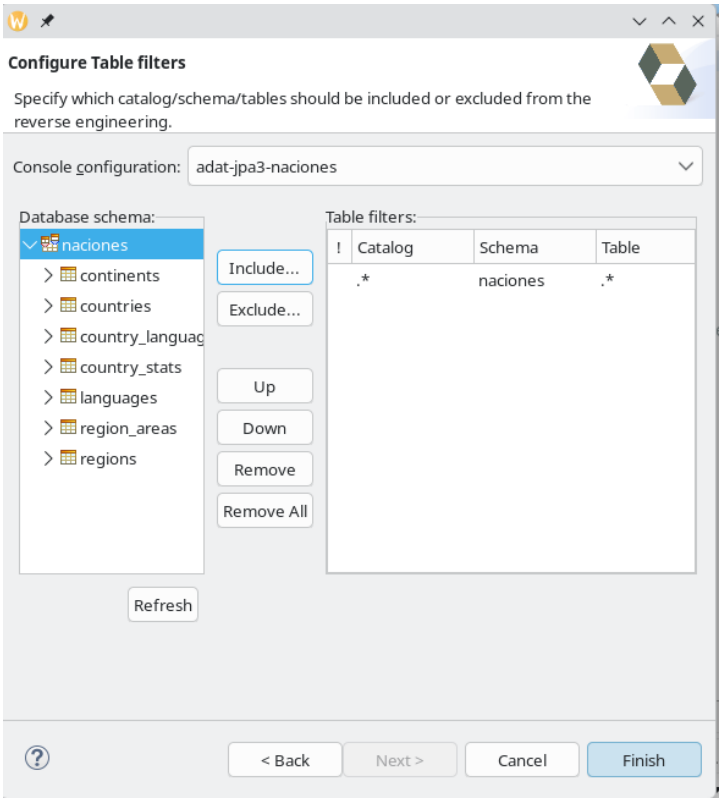
En la ventana siguiente, Console Configuration, dejamos todas las opciones por defecto. Atento al nombre que ha puesto automáticamente para el fichero de Console Configuration porque nos hará falta en el siguiente paso. Podemos corregirlo nosotros si queremos. En mi caso el nombre que he elegido es adat-jpa3-naciones

Ahora creamos el fichero XML Hibernate Reverse Engineering, otra vez desde:

New->Other->Hibernate

Lo crearemos igualmente en el directorio src y mantendremos el nombre sugerido por defecto (hibernate.reveng.xml)

Elegimos el fichero de configuración de consola que creamos en el paso anterior y pulsamos el botón de Refresh. A la izquierda debería de aparecer nuestra base de datos y, desplegando, nos aparecerán las tablas:

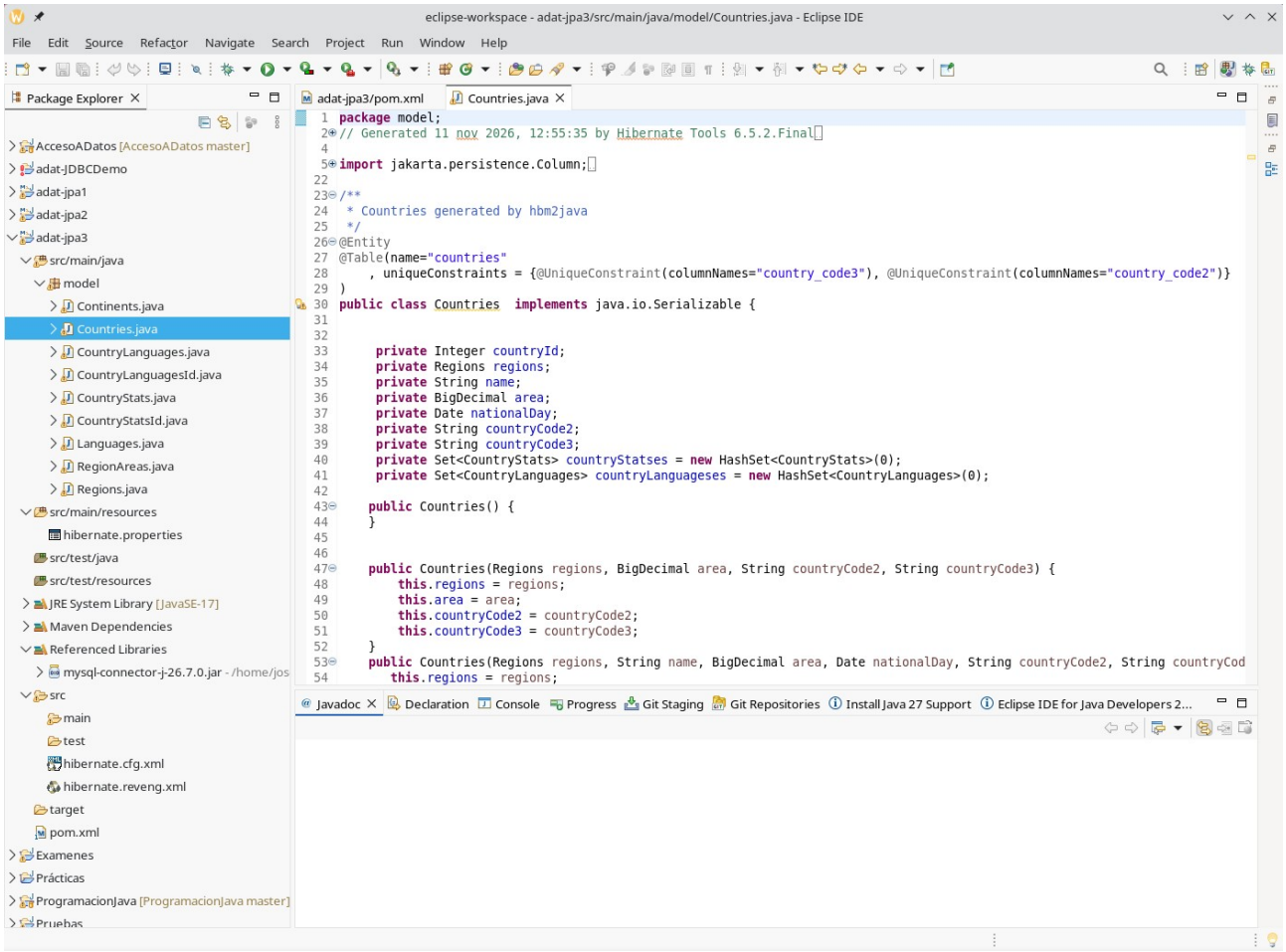


Podemos seleccionar individualmente las tablas con las que vamos a trabajar o la base de datos completa y luego pulsar include. Nuestra selección pasará a la ventana de la izquierda.

NOTA: No puedes seleccionar tablas por grupo y pulsar Include. Tiene un bug que hace que solo incluya la primera de la selección. Tienes que seleccionar la base de datos completa o las tablas una a una e ir pulsando Include

NOTA: Cualquier edición posterior a estos archivos la podemos hacer manualmente en la carpeta src donde estarán ambos.

Al terminar este proceso nos debería de generar un paquete con las clases de cada tabla y los getters y setters correspondientes



La generación se realiza de forma automática debido al bloque executions que hemos puesto en el pom.xml:

```
<executions>
	<execution>
		<id>generate-entities</id>
		<phase>generate-sources</phase>
		<goals>
			<goal>hbm2java</goal>
		</goals>
	</execution>
</executions>
```

Este bloque puede suponer un problema si modificamos las clases puesto que haría que se pudieran reescribir nuestros cambios. Solución: borrar o comentar el bloque anterior una vez generadas las clases o borrarlo de nuestra configuración inicial y lanzar manualmente la generación de las clases.

Para lanzar manualmente esta generación pulsamos sobre el nombre del proyecto y con el botón derecho elegimos:

Run As → Maven build

En la ventana resultante escribimos en Goals lo siguiente:

hibernate-tools:hbm2java

Pulsamos Apply y Run y se nos deberían de generar las clases.

NOTA: Si no aparecen de forma inmediata ponte sobre la carpeta del package model y pulsa F5

NOTA: Si nuestras tablas son muy complejas el código generado presentará muchos problemas porque las anotaciones que genera son obsoletas y no nos permiten trabajar con las versiones modernas de hibernate. He probado con pokemondb, por ejemplo, y presenta problemas graves que habría que resolver manualmente

Si abres, por ejemplo, la entidad Countries verás que la clave primaria está ahora con estas anotaciones:

```
@Id
@GeneratedValue(strategy=IDENTITY)
```

IDENTITY cumple la misma función que GenerationType.IDENTITY que habíamos visto antes: es una clave autoincrementable que gestiona directamente el gestor de bases de datos

Para continuar, tenemos que generar un archivo persistence.xml y grabarlo dentro de una carpeta llamada src/main/resources/META-INF:

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence xmlns="https://jakarta.ee/xml/ns/persistence"
             xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
             xsi:schemaLocation="https://jakarta.ee/xml/ns/persistence
https://jakarta.ee/xml/ns/persistence/persistence_3_1.xsd"
             version="3.1">

    <persistence-unit name="clasePU">
      <!-- Aquí tenemos que listar todas las clases que usemos -->
        <class>model.Countries</class>

        <properties>
            <property name="jakarta.persistence.jdbc.url"
value="jdbc:mariadb://localhost:3306/naciones"/>
            <property name="jakarta.persistence.jdbc.user" value="profe"/>
            <property name="jakarta.persistence.jdbc.password" value="abc123"/>

            <!-- Crea/actualiza las tablas a partir de las entidades -->
            <property name="hibernate.hbm2ddl.auto" value="update"/>
            <!-- Muestra el SQL que genera Hibernate -->
            <property name="hibernate.show_sql" value="false"/>
```

```
            <property name="hibernate.format_sql" value="true"/>
        </properties>
    </persistence-unit>
</persistence>
```

Yo he puesto solo la entidad Countries porque es la única que voy a usar en este ejemplo. Si quisiéramos usar alguna mas tendríamos que definirla ahí también. Hazlo porque vamos a usar alguna tabla mas en los ejemplos.

Otra cosa que podemos corregir: puesto que los nombres de las entidades se crean directamente de las tablas vemos clases en plural (Countries, Regions…) que no cuadran con las convenciones de POO. Hibernate ha pasado de minúsculas a mayúsculas (eso es correcto) pero no es capaz de distinguir entre singulares y plurares. Yo no lo he hecho en los ejemplos que vienen a continuación, pero no estaría de mas que tú lo hicieras

Y empezamos a trabajar…

Por último, creamos otro package (yo lo he llamado main) dentro de src/main/java y en el creamos nuestra clase Main con este contenido que, como ves, es muy similar al que usamos en el primer tutorial de esta serie:

```
package main;

import java.util.List;
import java.util.logging.Level;
import java.util.logging.Logger;

import jakarta.persistence.EntityManager;
import jakarta.persistence.EntityManagerFactory;
import jakarta.persistence.Persistence;
import model.*;

public static void main(String[] args) {
            LOG_HIBERNATE.setLevel(Level.SEVERE);

            EntityManagerFactory emf =
Persistence.createEntityManagerFactory("clasePU");
            EntityManager em = emf.createEntityManager();

            try {
                  listadoNombresPaises(em);
            } catch (Exception e) {
                  System.out.println("Error: " + e.getMessage());
            }
            em.close();
            emf.close();
      }
```

```
public static void listadoNombresPaises(EntityManager em){
            String jpql = "SELECT pais.name FROM Countries pais";
            List<String> listaNombres = em.createQuery(jpql,
String.class).getResultList();
            System.out.println("--- Nombres de todos los paises ---");
            for (String nombre : listaNombres) {
                  System.out.println(nombre);
                  }
            System.out.println("Total de paises: " + listaNombres.size());
      }
```

En el anterior ejemplo, como ves, se listan todos los nombres de pais de la tabla Countries. No te olvides del import model.* Puesto que nuestras Entidades están en un paquete diferente es necesario hacerlo así.

Vamos a ver ahora un método que nos lista los idiomas que se hablan en un país usando una consulta multitabla y sin tener que usar JPQL:

```
public static void verIdiomas(EntityManager em, String pais) {
      String jpql = "FROM Countries WHERE name = :pais";
      List<Countries> listaPaises = em.createQuery(jpql, Countries.class)
                  .setParameter("pais", pais)
                  .getResultList();
      if (listaPaises.isEmpty())
            System.out.println("No existe ese país");
      else if (listaPaises.size()!= 1)
            System.out.println("Hay mas de un país con el mismo nombre");
      else {
            Countries miPais = (Countries)listaPaises.get(0);
            Set<CountryLanguages> listaIdiomas = miPais.getCountryLanguageses();
            if(listaIdiomas.size()!=0) {
                  System.out.println("--- Idiomas de " + pais + " ---");
                  for(CountryLanguages fila: listaIdiomas)
                        System.out.println(fila.getLanguages().getLanguage());
            }
            else
                  System.out.println("No hay idiomas para " + pais);
      }
}
```

Como habrás visto si observas las clases que se han generado, Hibernate a creado una entidad llamada CountryLanguagesId que no existe en la base de datos. Esta tabla se crea puesto que CountryLanguages tiene una clave compuesta pero no tenemos que usarla directamente para nada. Hibernate se encarga de gestionarla. Si miras en esa clase, de todas formas, verás dos métodos que se llaman equals y hash. Hibernate los usa también internamente para hacer comparaciones entre los campos clave.

A nosotros lo que nos interesa es la clase CountryLanguages. A partir de un pais llamamos al método getCountryLangaugeses y obtenemos una lista de todos los idiomas que se hablan en ese país sin necesidad de hacer otro query.

Si observamos ese método:

```
@OneToMany(fetch=FetchType.LAZY, mappedBy="countries")
    public Set<CountryLanguages> getCountryLanguageses() {
        return this.countryLanguageses;
    }
```

Vemos, por un lado, que la relación está establecida como OneToMany (uno a muchos). Un pais puede tener muchos idiomas.

Vemos también un atributo que se llama fetch definido como FetchType.LAZY Eso indica que es una relación perezosa y significa que los elementos solo se cargaran de la base de datos en el momento en el que se necesiten. Lo contrario sería FetchType.EAGER que cargaría inmediatamente los elementos de la base de datos. Genera código mas rápido pero también que consume mucha mas memoria. Por último, vemos mappedBy = "countries". Eso le indica a Hibernate que el atributo con el que se establece la relación en la entidad CountryLanguages se llama countries

Vamos a ver ahora una consulta multitabla pero usando JPQL:

```
private static void paisesDeUnContinente(EntityManager em, String continente) {
      String jpql = "SELECT c FROM Countries c "
                  + "JOIN c.regions r "
                  + "JOIN r.continents co "
                  + "WHERE co.name = :continente "
                  + "ORDER BY c.name";
      List<Countries> paises = em.createQuery(jpql, Countries.class)
                  .setParameter("continente", continente)
                  .getResultList();

      System.out.println("--- Paises de " + continente + " ---");
      for (Countries pais : paises) {
            System.out.println(pais.getName());
      }
}
```